1. Jaka była łączna suma transakcji (SalesOrderHeader.SubTotal) w poszczególnych latach dla kolejnych dni tygodnia?

   Od roku 2022 całkowita wartość sprzedaży rośnie natomiast w 2025 roku maleje ona drastycznie, może być to spowodowane niepełnymi danymi za ten okres. W różnych latach najlepszy dzień tygodnia zmienia się.

```sql
SELECT YEAR(OrderDate)              AS OrderYear,
   DATENAME(WEEKDAY, OrderDate) AS DayOfWeek,
   SUM(SubTotal)                AS YearlyDailyTotal
    FROM Sales.SalesOrderHeader
    GROUP BY YEAR(OrderDate), DATENAME(WEEKDAY, OrderDate), DATEPART(WEEKDAY, OrderDate)
    ORDER BY OrderYear, DATEPART(WEEKDAY, OrderDate);
```

2. Zaproponuj podział klientów na 3 rozłączne grupy wiekowe. Ilu różnych klientów dokonało zakupów w kolejnych miesiącach roku w każdej z grup? Ilu klientów w poszczególnych grupach wykonało zakup dokładnie jeden raz?

   W przedziale 0-30 nie ma ani jednego kupującego, w przedziale 30 do 70 najwięcej unikalnych klientów kupuje w okresie marca, kwietnia i maja, co jest również zauważalne w grupie 70+ natomiast mniej.
   Z CTE:

```sql
WITH CustomerAge AS (SELECT c.CustomerID,
                            DATEDIFF(YEAR, pd.BirthDate, GETDATE()) AS Age,
                            CASE
                                    WHEN DATEDIFF(YEAR, pd.BirthDate, GETDATE()) < 30 THEN '0-30'
                                    WHEN DATEDIFF(YEAR, pd.BirthDate, GETDATE()) BETWEEN 30 AND
70 THEN '30-70'
                                    ELSE '70+'
                                    END                                AS AgeGroup
                        FROM Sales.Customer c
                                JOIN Sales.vPersonDemographics pd ON c.PersonID =
pd.BusinessEntityID),

     PurchaseCoutnt AS (SELECT CustomerID, COUNT(SalesOrderID) TotalPurchases
                        FROM Sales.SalesOrderHeader
                        GROUP BY CustomerID)
SELECT ca.AgeGroup,
       DATENAME(MONTH, soh.OrderDate)                        OrderMonth,
       COUNT(DISTINCT soh.CustomerID)                        UniqueCustomer,
       SUM(CASE WHEN pc.TotalPurchases > 1 THEN 0 ELSE 1 END) SinglePurchase
FROM Sales.SalesOrderHeader soh
         JOIN CustomerAge ca ON soh.CustomerID = ca.CustomerID
         JOIN PurchaseCoutnt pc ON soh.CustomerID = pc.CustomerID
GROUP BY ca.AgeGroup, DATENAME(MONTH, soh.OrderDate), DATEPART(MONTH, soh.OrderDate)
```

```
ORDER BY AgeGroup, DATEPART(MONTH, soh.OrderDate);
```

MemoryGrantInfo	
DesiredMemory	32600
GrantedMemory	32600
GrantWaitTime	0
IsMemoryGrantFeedback	NoFirstExecution
LastRequestedMemory	0
MaxQueryMemory	903248
MaxUsedMemory	5376
RequestedMemory	32600
RequiredMemory	3584
SerialDesiredMemory	32600
SerialRequiredMemory	3584
NonParallelPlanReason	TSQLUserDefinedFunctionsNotPa
QueryTimeStats	
CpuTime	542
ElapsedTime	542

Bez CTE:

```
SELECT ca.AgeGroup,
       DATENAME(MONTH, soh.OrderDate) AS OrderMonth,
       COUNT(DISTINCT soh.CustomerID) AS UniqueCustomer,
       SUM(CASE WHEN pc.TotalPurchases > 1 THEN 0 ELSE 1 END) AS SinglePurchase
FROM Sales.SalesOrderHeader soh
JOIN (
    SELECT c.CustomerID,
           DATEDIFF(YEAR, pd.BirthDate, GETDATE()) AS Age,
           CASE
               WHEN DATEDIFF(YEAR, pd.BirthDate, GETDATE()) < 30 THEN '0-30'
               WHEN DATEDIFF(YEAR, pd.BirthDate, GETDATE()) BETWEEN 30 AND 70 THEN '30-70'
               ELSE '70+'
           END AS AgeGroup
    FROM Sales.Customer c
    JOIN Sales.vPersonDemographics pd ON c.PersonID = pd.BusinessEntityID
) ca ON soh.CustomerID = ca.CustomerID
JOIN (
    SELECT CustomerID,
           COUNT(SalesOrderID) AS TotalPurchases
    FROM Sales.SalesOrderHeader
    GROUP BY CustomerID
) pc ON soh.CustomerID = pc.CustomerID
GROUP BY ca.AgeGroup, DATENAME(MONTH, soh.OrderDate), DATEPART(MONTH, soh.OrderDate)
    ORDER BY ca.AgeGroup, DATEPART(MONTH, soh.OrderDate);
```

MemoryGrantInfo	
DesiredMemory	32600
GrantedMemory	32600
GrantWaitTime	0
IsMemoryGrantFeedback	NoFirstExecution
LastRequestedMemory	0
MaxQueryMemory	926760
MaxUsedMemory	5376
RequestedMemory	32600
RequiredMemory	3584
SerialDesiredMemory	32600
SerialRequiredMemory	3584
NonParallelPlanReason	TSQLUserDefinedFunctionsNotPa

QueryTimeStats	
CpuTime	577
ElapsedTime	577

Przy użyciu CTE czas wykorzystania procesora jest nieco mniejszy.

3. Przygotuj zestawienie produktów, których sprzedaje się miesięcznie min. 20 sztuk. Dla każdego produktu podaj jego kategorię.

   Najwięcej sprzedaje się butelek na wodę, czapek i kamizelek. Kategorie akcesoriów i ubrań zdecydowanie przewyższają inne kategorie pod względem ilości sprzedanych sztuk produktów.

   Z CTE:

```
WITH MonthlyProductSales AS (
SELECT p.Name                 ProductName,
       pc.Name                CategoryName,
       YEAR(soh.OrderDate)    SalesYear,
       MONTH(soh.OrderDate)   SalesMonth,
       SUM(sod.OrderQty)      TotalQuantity
FROM Production.Product p
         JOIN Production.ProductSubcategory psc ON p.ProductSubcategoryID =
psc.ProductSubcategoryID
         JOIN Production.ProductCategory pc ON psc.ProductCategoryID = pc.ProductCategoryID
         JOIN Sales.SalesOrderDetail sod ON p.ProductID = sod.ProductID
         JOIN Sales.SalesOrderHeader soh ON sod.SalesOrderID = soh.SalesOrderID
GROUP BY p.Name,
         pc.Name,
         YEAR(soh.OrderDate),
         MONTH(soh.OrderDate))
SELECT ProductName,
       CategoryName,
       SalesYear,
       SalesMonth,
       TotalQuantity
FROM MonthlyProductSales
WHERE TotalQuantity >= 20
ORDER BY SalesYear DESC, SalesMonth DESC, TotalQuantity DESC;
```

MemoryGrantInfo	
DesiredMemory	10320
GrantedMemory	10320
GrantWaitTime	0
IsMemoryGrantFeedb	YesPercentileAdjusting
LastRequestedMemor	8984
MaxQueryMemory	850920
MaxUsedMemory	2904
RequestedMemory	10320
RequiredMemory	4608
SerialDesiredMemory	10320
SerialRequiredMemor	4608

QueryTimeStats	
CpuTime	182
ElapsedTime	285

Bez CTE:

```
SELECT p.Name                AS ProductName,
       pc.Name               AS CategoryName,
       MONTH(soh.OrderDate)  AS Month,
       YEAR(soh.OrderDate)   AS Year,
       SUM(sod.OrderQty)     AS TotalQty
        FROM Production.Product p
```

```
        INNER JOIN Production.ProductSubcategory psc ON p.ProductSubcategoryID =
psc.ProductSubcategoryID
        INNER JOIN Production.ProductCategory pc ON psc.ProductCategoryID =
pc.ProductCategoryID
        INNER JOIN Sales.SalesOrderDetail sod ON p.ProductID = sod.ProductID
        INNER JOIN Sales.SalesOrderHeader soh ON sod.SalesOrderID = soh.SalesOrderID
GROUP BY p.Name, pc.Name, YEAR(soh.OrderDate), MONTH(soh.OrderDate)
HAVING SUM(sod.OrderQty) >= 20
ORDER BY Year, Month, TotalQty DESC;
```

MemoryGrantInfo	
DesiredMemory	12968
GrantedMemory	12968
GrantWaitTime	0
IsMemoryGrantFeedb	YesPercentileAdjusting
LastRequestedMemor	12968
MaxQueryMemory	831800
MaxUsedMemory	2904
RequestedMemory	12968
RequiredMemory	4608
SerialDesiredMemory	12968
SerialRequiredMemor	4608

QueryTimeStats	
CpuTime	155
ElapsedTime	303

Czas wykonania był gorszy przy użyciu CTE natomiast maksymalne zużycie pamięci było takie samo

4. Przygotuj zestawienie, w którym przeanalizujesz, ilu jest różnych klientów dla każdej płci w kolejnych miesiącach (05.2011 – 06.2024)? Jak procentowo rozkłada się ich udział w całkowitej wartości sprzedaży (Sales.SalesOrderHeader.TotalDue)?

   Największy udział w sprzedaży mają zakupy w których płeć nabywcy nie została zaklasyfikowana, natomiast w pozostałych przypadkach podział jest równomierny.

```
SELECT pd.Gender,
       MONTH(soh.OrderDate)            OrderMonth,
       YEAR(soh.OrderDate)             OrderYear,
       COUNT(DISTINCT soh.CustomerID) CustomerCount,
       SUM(soh.TotalDue)               MonthlyTotalDue,
       SUM(soh.TotalDue) * 100.0 /
       SUM(SUM(soh.TotalDue)) OVER ()
                                          PercentOfMonthlyTotal
FROM Sales.SalesOrderHeader soh
        INNER JOIN Sales.Customer c ON soh.CustomerID = c.CustomerID
        INNER JOIN Sales.vPersonDemographics pd ON c.PersonID = pd.BusinessEntityID
WHERE soh.OrderDate BETWEEN '2011-05-01' AND '2024-06-30'
GROUP BY pd.Gender, YEAR(soh.OrderDate), MONTH(soh.OrderDate)
ORDER BY OrderYear, OrderMonth, pd.Gender;
```

5. Przeanalizuj udział sprzedanych produktów w poszczególnych podkategoriach w stosunku do całych kategorii (zarówno pod względem liczbowym jak i wartościowym).

   Największy udział w liczbie sprzedaży mają dętki, ale nie przekłada się to na udział w wartości sprzedaży, przy 22% udziale kaski odpowiadają za prawie 40% przychodu w kategorii akcesoriów. W innych kategoriach najlepiej sprzedają się rowery miejskie,

koszulki, oraz ramy rowerowe.

```
SELECT pc.Name
Category,
   psc.Name
Subcategory,
   COUNT(sod.SalesOrderDetailID) * 1.0 /
   SUM(COUNT(sod.SalesOrderDetailID)) OVER (PARTITION BY pc.Name) * 100
QtySharePercent,
   SUM(sod.LineTotal) / SUM(SUM(sod.LineTotal)) OVER (PARTITION BY pc.Name) * 100
ValueSharePercent
FROM Production.ProductCategory pc
     INNER JOIN Production.ProductSubcategory psc ON pc.ProductCategoryID =
psc.ProductCategoryID
     INNER JOIN Production.Product p ON psc.ProductSubcategoryID = p.ProductSubcategoryID
     INNER JOIN Sales.SalesOrderDetail sod ON p.ProductID = sod.ProductID
GROUP BY pc.Name, psc.Name;GROUP BY pc.Name, psc.Name;
```

6. Przygotuj zestawienie, w którym możliwa będzie analiza regionalna z uwzględnieniem lokalnej waluty (kwoty sprzedaży w zależności od waluty i regionu).

   Największy przychód został odnotowany w regionach Stanów Zjednoczonych, Kanadzie i Wielkiej Brytanii.

```
SELECT
        st.Name AS Region,
        cur.Name AS Currency,
        SUM(soh.TotalDue) AS TotalInLocalCurrency,
        SUM(soh.TotalDue / ISNULL(scr.EndOfDayRate, 1)) AS TotalInUSD
    FROM Sales.SalesOrderHeader soh
    JOIN Sales.SalesTerritory st ON soh.TerritoryID = st.TerritoryID
    LEFT JOIN Sales.CurrencyRate scr ON soh.CurrencyRateID = scr.CurrencyRateID
    LEFT JOIN Sales.Currency cur ON scr.ToCurrencyCode = cur.CurrencyCode
    GROUP BY st.Name, cur.Name
           ORDER BY Region;
```